A 1.3.1-5 Operating Systems



A 1.3 Operating Systems and Control Systems

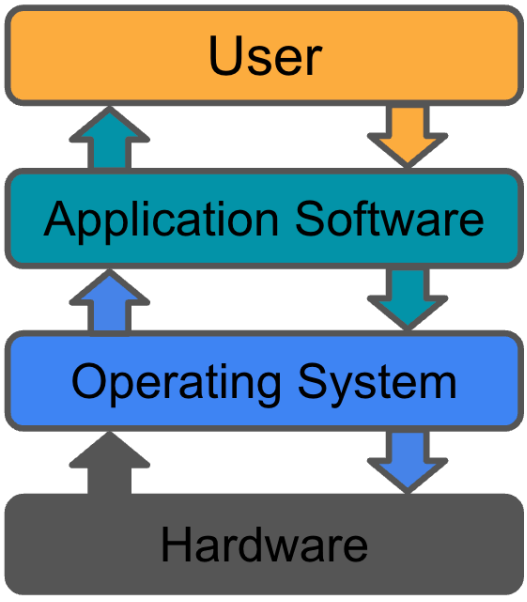
This section will delve into the fascinating world of operating systems (OS), the unsung heroes that make your computer work! We'll explore their roles, functions, and how they manage the complex dance of hardware and software.

A1.3.1 What is the Role of an Operating System?

Imagine an orchestra with all its different instruments. Without a conductor, it would be chaos! The operating system is like the conductor of your computer, ensuring all the different parts work together harmoniously.

Managing System Resources: The OS manages essential resources like the CPU (the brain), memory (the workspace), and storage (the filing cabinet). It decides which program gets to use the CPU, how much memory each program can have, and where files are stored.
Abstracting Hardware Complexities: The OS hides the messy details of how the hardware works. You don't need to know the intricate commands to read a file from the hard drive – the OS takes care of that for you. This abstraction makes it much easier to use a computer and write programs.

Key Question: Can you think of an analogy for how the OS abstracts hardware complexities? (Hint: Think of a translator or a waiter in a restaurant.)



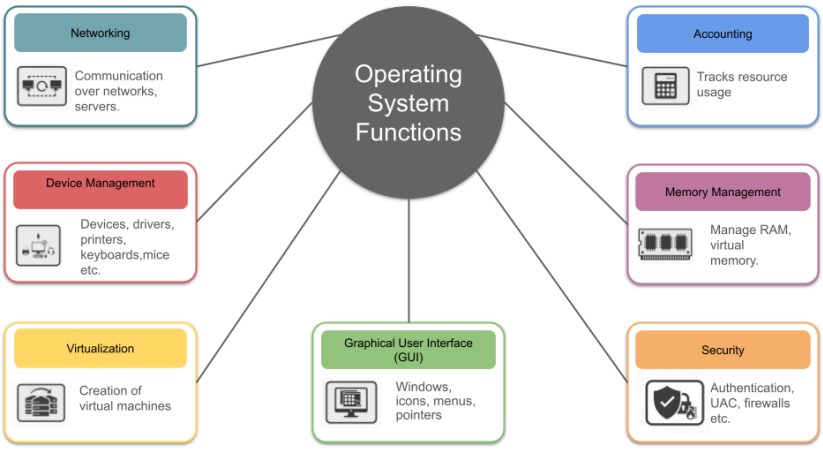
A layered diagram showing the OS sitting between the user/applications and the hardware.

A1.3.2 What Functions Does an Operating System Perform?

The OS is a multi-tasker, juggling many functions to keep your computer running smoothly. Some of its key responsibilities include:

Memory Management: The OS efficiently allocates memory to different programs, ensuring they have enough space to run without interfering with each other. It uses techniques like virtual memory to create the illusion of more memory than is physically available.
Device Management: The OS controls all the devices connected to your computer, from your keyboard and mouse to your printer and monitor. It uses special software called drivers to communicate with each device.
Security: The OS protects your computer from unauthorized access and malicious software. It uses features like user authentication (passwords, biometrics), access control, and firewalls to keep your data safe.
Accounting: The OS tracks how different users and programs use resources. This information can be used to analyze performance, ensure fair resource allocation, and even for billing purposes in multi-user systems.
Graphical User Interface (GUI): The OS provides a user-friendly interface that allows you to interact with your computer using windows, icons, menus, and pointers.
Virtualization: The OS can create virtual machines, allowing you to run multiple operating systems on a single physical computer. This is useful for server consolidation, software testing, and cloud computing.
Networking: The OS enables your computer to communicate with other computers over a network. It handles the complex tasks of sending and receiving data packets, ensuring they reach their destination reliably.

Key Question: Which of these functions do you think is most important for a smooth user experience, and why?



The different functions of an operating system.

A1.3.3 How Do Approaches to Scheduling Differ?

The OS is responsible for scheduling processes and deciding which program gets to use the CPU and for how long. This is like a traffic controller managing the flow of cars on a busy road. Different scheduling algorithms have different advantages and disadvantages:

First-Come, First-Served (FCFS): Processes are executed in the order they arrive. Simple, but can lead to long wait times for shorter processes.
Round Robin: Each process gets a fixed time slice on the CPU. It is fairer than FCFS, but it can have overhead from frequent context switching.
Multilevel Queue Scheduling: Processes are assigned to different queues based on their priority or resource needs, which allows for prioritizing important tasks.
Priority Scheduling: Processes with higher priority are given preference. This is useful for real-time systems and tasks that require immediate attention.

Key Question: Can you imagine a real-world scenario where each scheduling algorithm would be most suitable?

Scheduling algorithm	CPU Overhead	Turnaround time	Throughput	Response time
First In First Out	Low	High	Low	Low
Shortest Job First	Medium	Medium	High	Medium
Priority based scheduling	Medium	High	Low	High
Round-robin scheduling	High	Medium	Medium	High
Multilevel Queue scheduling	High	Medium	High	Medium

A table showing how each scheduling algorithm works.

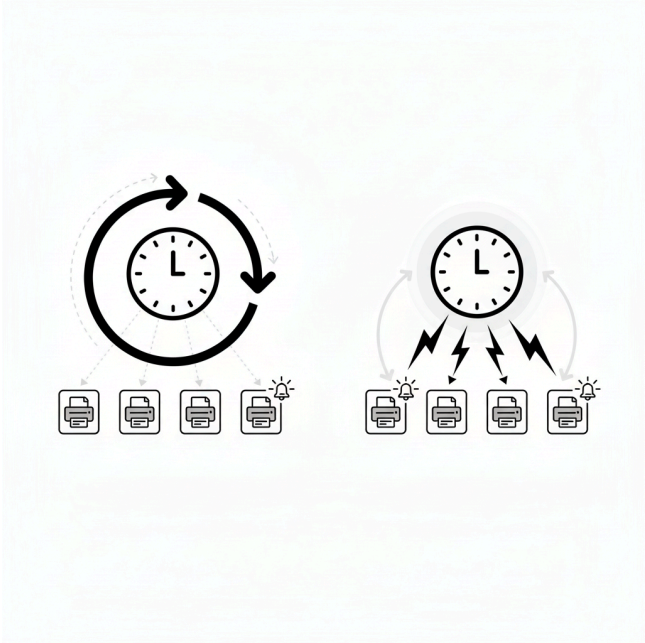
A1.3.4 How Do Polling and Interrupt Handling Compare?

The OS must handle input/output (I/O) operations from devices like keyboards, mice, and network cards. There are two main approaches:

Polling: The OS constantly checks the device to see if it has any data or requests. This can be inefficient if the device rarely needs attention.
Interrupt Handling: The device sends an interrupt (a signal) to the OS when it needs attention. This allows the OS to focus on other tasks when the device is idle.
The choice between polling and interrupts depends on factors like:

- **Event Frequency:** How often does the device need attention?
- **CPU Overhead:** The impact of polling on CPU usage.
- **Power Source:** There is a need to conserve battery life in portable devices.
- **Event Predictability:** Whether events are predictable or unpredictable.
- **Controlled Latency:** The need for immediate response in real-time systems.
- **Security Concerns:** There is a need for tighter control in security-sensitive operations.

Key Question: Why might interrupt handling be preferred for a keyboard, while polling might be more suitable for a network connection?



An image comparing polling (a loop checking the device) with interrupt handling (a signal from the device).

A1.3.5 What is the Role of the OS in Managing Multitasking and Resource Allocation? (HL only)

Multitasking allows your computer to seemingly run multiple programs at the same time.

The Illusion of Parallelism: Most computers have a single CPU, yet they seem to run multiple programs simultaneously. This is multitasking, where the OS rapidly switches between tasks, giving the illusion of parallel execution.
Resource Allocation: The OS acts as a master coordinator, carefully allocating resources (CPU time, memory, I/O devices) to different tasks to ensure they all progress.

The OS faces several challenges in managing this:

Context Switching: Saving and restoring the state of different tasks when switching between them.
Resource Contention: Managing competition for resources like memory and I/O devices.
Synchronisation: Ensuring that tasks that depend on each other execute in the correct order.
Deadlock: Preventing situations where two or more tasks are blocked indefinitely, each waiting for the other to release a resource.

Key Question: What four conditions must be present for a deadlock to occur?



Operating system Quiz

1

Which of the following is NOT a core function of an operating system?

A. ◯ Running application software

B. ◯ Compiling source code

C. ◯ Providing a user interface

D. ◯ Managing system resources

2

What is the purpose of abstraction in an operating system?

A. ◯ To make the computer run faster

B. ◯ To prevent software crashes

C. ◯ To enable multitasking

D. ◯ To hide complex hardware details from users and applications

3

Which scheduling algorithm gives each process a fixed time slice on the CPU?

A. ○ Round Robin

B. ○ First-Come, First-Served

C. ○ Multilevel Queue Scheduling

D. ○ Priority Scheduling

4

What is a device driver?

A. ○ A type of scheduling algorithm

B. ○ A security feature that prevents unauthorized access

C. ○ A hardware component that controls a device

D. ○ A software program that allows the OS to communicate with a device

5

Which of the following is an advantage of virtualization?

A. ○ Decreased flexibility

B. ○ Reduced security

C. ○ Increased hardware costs

D. ○ Improved resource efficiency

6

What is the main difference between polling and interrupt handling?

A. ○ Polling is used in real-time systems, while interrupt handling is used in embedded systems

B. ○ Polling is more efficient than interrupt handling

C. ○ Interrupt handling is only used for keyboards and mice

D. ○ Polling involves constantly checking a device, while interrupt handling waits for a signal from the device

7

Which of the following is NOT a factor that influences the choice between polling and interrupts?

A. ○ The color of the device

B. ○ Event frequency

C. ○ Power source

D. ○ CPU overhead

8

What is deadlock?

A. ○ A situation where a program crashes

B. ○ A type of scheduling algorithm

C. ○ A situation where two or more processes are blocked indefinitely, each waiting for the other to release a resource

D. ○ A security vulnerability

9

Which of the following is a strategy for preventing deadlock?

A. ○ Disabling interrupts

B. ○ Ignoring resource requests

C. ○ Randomly assigning resources

D. ○ Resource ordering

10

What is the role of a transducer in a control system?

A. ○ To carry out commands from the controller

B. ○ To gather information about the environment

C. ○ To make decisions based on sensor input

D. ○ To convert one form of energy to another